



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: Camila Raquel Marcos Medinilla **Carné:** 1120525

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Es un estilo de arquitectura que se utiliza para la comunicación e intercambio de datos entre dos sistemas web. Usa recursos que se identifican con IDs.

2. ¿Qué es un endpoint?

Es una ruta específica de una URL a la que se accede para interactuar con un recurso determinado dentro de la API. Es la dirección exacta con la que el cliente conecta para realizar una acción o solicitar datos.

3. ¿Qué diferencia existe entre una URL y un endpoint?

La URL es la dirección completa y un endpoint es una sección específica de un URL.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

Los métodos HTTP indican al servidor qué acción se quiere realizar sobre un recurso de la API.

5. ¿Qué es JSON y por qué es común en las APIs?

JavaScript Object Notation es un formato de texto utilizado para estructurar y transferir datos. Es el estándar más común en las APIs porque es fácil de leer y escribir tanto para los humanos como para las máquinas. Y es compatible con los lenguajes de programación modernos.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es un número enviado por el servidor que indica el resultado de la petición realizada por el cliente.

200 significa OK o éxito.

404 significa que no se pudo encontrar el recurso solicitado (not found)

500 significa que hubo un error del servidor.

7. ¿Qué información se puede encontrar en la respuesta de una API?

Código de estado HTTP: 200, 404, 500

Encabezados: que proveen información sobre la respuesta como tipo de contenido o fecha.

Cuerpo: los datos reales solicitados, usualmente en formato JSON.

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

Permite diseñar y validar la lógica de la API sin necesidad de escribir código de cliente. Permite verificar rápidamente si las rutas responden como se espera.

9. ¿Qué es y para qué sirve Swagger UI?

Es una herramienta visual que genera documentación interactiva para una API a partir de su especificación. Sirve para que los desarrolladores entiendan cómo funciona la API, qué parámetros requiere cada ruta, qué tipo de respuestas devuelve, e incluso permite probar los endpoints directamente desde el navegador.



10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Postman actúa como un entorno completo de desarrollo para APIs.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.
- Ejecute la solicitud con Try it out y registre el código HTTP recibido.
- Explique con sus palabras qué información contiene la respuesta.

Evidencia: captura de Swagger mostrando la operación y su respuesta.

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

El JSON debe incluir, como mínimo:

id
name
status
category
tags

Registre el ID utilizado y verifique la respuesta. Luego consulte nuevamente la mascota por medio de GET /pet/{petId}.

Evidencia: JSON enviado + respuesta de creación + consulta posterior.

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y



- ¿Qué método HTTP utilizó y por qué?
- ¿Qué datos fueron enviados en el body?
- ¿Qué código HTTP devolvió la API?
- ¿Cómo comprobó que el cambio realmente ocurrió?

5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.
- Identifique el parámetro utilizado.
- Explique dónde se envía ese parámetro.
- Observe la estructura de la respuesta.
- Indique qué sucede si cambia el valor del parámetro por pending.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID
- POST – Crear mascota
- PUT – Modificar mascota
- GET – Buscar por status
- DELETE – Eliminar mascota
- En cada solicitud configure manualmente método, URL, parámetros, headers y body cuando corresponda. No utilice únicamente la opción de importar una colección.

7. Prueba de errores

Una API también debe analizarse cuando algo sale mal. Realice al menos tres pruebas que generen respuestas diferentes a una solicitud exitosa.

Ejemplos de escenarios:

1. Consultar un ID inexistente.
2. Enviar un JSON incorrecto o incompleto.
3. Utilizar un parámetro con un valor no esperado.
4. Intentar eliminar un recurso que ya no existe.

Para cada prueba registre:

- a) Solicitud realizada.
- b) Código HTTP recibido.
- c) Respuesta de la API.
- d) Qué esperaba que ocurriera.
- e) Qué ocurrió realmente.
- f) Explicación de la causa

8. API Detective



Suponga que usted recibe el siguiente requerimiento de un sistema:

“El sistema debe registrar una mascota, consultar su información, actualizar su estado y finalmente eliminarla. Si el recurso no existe, el sistema debe poder identificarlo mediante la respuesta de la API.”

Resuelva el reto:

- Defina qué endpoint utilizaría para cada operación.
- Defina el método HTTP correspondiente.
- Indique qué respuesta debe analizar el programa.
- Ejecute el flujo completo en Postman.

Capture evidencia de cada paso.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab1/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las pruebas realizadas.

Enviar link de github a plataforma de la universidad.